

ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Midterm - Week 7 (Control Flow)

Instructor: Henry V. Dela Cruz

I. Overview and Objectives

This week marks a major milestone. Our programs will no longer just execute top-to-bottom in a straight line. We will introduce "Control Flow"—the ability for your program to make decisions (Conditionals) and repeat actions (Loops) based on live data.

Intended Learning Outcomes (ILOs):

- Define and utilize Python Data Types and apply Type Conversion explicitly.
- Implement various Python Operators (Arithmetic, Comparison, and Logical).
- Construct Conditional Statements (if, elif, else) to dictate program flow.
- Design Loop structures (while, for) and apply Loop Control Statements (break, continue).

II. Data Types and Type Conversion

Data types dictate what operations can be performed on a variable. Python handles four primary primitive types: `int` (integer), `float` (decimal), `str` (string), and `bool` (boolean: `True/False`).

Type Conversion (Casting) is the process of converting one data type to another. It is crucial when dealing with `input()`, which always returns a `str`.

```
# Explicit Type Conversion
raw_input = "25"           # This is a String
converted_num = int(raw_input) # Casted to an Integer
decimal_num = float(raw_input) # Casted to a Float (25.0)

# Converting back to string for concatenation
print("The number is " + str(converted_num))
```

III. Python Operators

Operators are special symbols that carry out arithmetic or logical computation. Beyond basic arithmetic (+, -, *, /, %), we must master Comparison and Logical operators to build conditional statements.

Comparison & Logical Operators

Operator Type	Symbols	Example	Result (if x=5)
Comparison	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	<code>x > 3</code>	True
Logical AND	<code>and</code>	<code>x > 3 and x < 10</code>	True
Logical OR	<code>or</code>	<code>x == 5 or x == 10</code>	True
Logical NOT	<code>not</code>	<code>not(x == 5)</code>	False

IV. Conditional Statements (if, elif, else)

Conditional statements allow your program to execute different blocks of code based on whether a condition evaluates to True or False. Notice the strict use of the colon : and **indentation** to define the block.

```
temperature = 38.5

if temperature >= 37.5:
    # Executes if condition is True
    print("Patient has a fever.")
elif temperature <= 35.0:
    # Executes if first condition is False, but this one is True
    print("Patient has hypothermia.")
else:
    # Executes if ALL above conditions are False
    print("Temperature is normal.")
```

V. Introduction to Loops

Loops allow us to execute a block of code multiple times. Python has two primitive loop commands: while loops and for loops.

1. The while Loop & Control Statements

A while loop executes a set of statements as long as a condition is true. We use **Loop Control Statements** to manage the loop's behavior:

- **break**: Stops the loop entirely, even if the condition is still true.
- **continue**: Skips the current iteration and jumps back to the top of the loop.

```
count = 0
while count < 5:
    count += 1
    if count == 3:
        continue # Skips printing '3'
    print(f"Processing file {count}...")

# Output: 1, 2, 4, 5 (3 is skipped)
```

Laboratory Coding Session: Bakawan Health Evaluator

Scenario: You are writing the diagnostic script for the "Protect Our Bakawan" water quality sensors. The program must evaluate a mangrove zone based on user input, output a health status, and then **ask the user if they want to evaluate another zone**, looping continuously until they type 'exit'.

Instructions (Your code must implement these exact rules):

1. Wrap your entire logic inside a while loop that keeps running.
2. Use `input()` to ask for the **Zone Name**, **Salinity Level (ppt)**, and **pH Level**.
3. Cast the Salinity and pH inputs to `float()`.
4. Use `if/elif/else` and Logical Operators (`and`, `or`):
 - If Salinity > 35 **or** pH < 6.5 → Print "CRITICAL: Immediate action required."
 - If Salinity is between 20-35 **and** pH is between 6.5-8.0 → Print "OPTIMAL: Health is stable."
 - Otherwise → Print "WARNING: Monitor closely."
5. After the evaluation, ask the user: "Evaluate another zone? (yes/exit): "
6. If the user types `exit`, use the `break` statement to terminate the loop and print "System Shutdown".

Sample Program Run (Expected Output):

```
Enter Zone Name: Sector A
Enter Salinity (ppt): 38.2
Enter pH Level: 7.0
Status -> CRITICAL: Immediate action required.

Evaluate another zone? (yes/exit): yes

Enter Zone Name: Sector B
Enter Salinity (ppt): 25.5
Enter pH Level: 7.2
Status -> OPTIMAL: Health is stable.

Evaluate another zone? (yes/exit): exit
System Shutdown.
```

Submit your completed .py file once your terminal behaves exactly like the loop shown above.